

Práctica de Repaso — Nivel Intermedio

Una aplicación completa por desafío · JavaScript + HTML · Sin jQuery ni APIs

Presentación

Esta práctica tiene **tres desafíos**. Cada uno es una pequeña aplicación web completa que te va a pedir usar, de forma natural, los mismos conceptos que trabajaste en los ejercicios de clase teórica: *if/else if/else*, *switch*, bucles *for*, *while* y *do...while*, acumuladores, contadores y búsqueda del mayor. La diferencia con los ejercicios de teoría es que acá esos conceptos tienen que aparecer dentro de funciones conectadas a una página HTML real, leyendo inputs y mostrando resultados en el DOM — exactamente como en el parcial.

Cada desafío indica qué concepto de teoría usa cada parte, para que puedas ubicarte. El trabajo es **individual** y puede hacerse en jsbin.com o en archivos locales.

Mapa de conceptos cubiertos

Concepto de teoría	Ejercicio original	Dónde aparece en esta práctica
if / else if / else	Ej. 1 — positivo/negativo	Los tres desafíos: clasificar resultados
Bucle for	Ej. 2 — números 1 al 10	Desafío 1: generar tabla; Desafío 3: recorrer lista
while + acumulador	Ej. 3 — suma 1 al 100	Desafío 1: calcular promedio
while + contador	Ej. 4 — contar pares	Desafío 1: contar aprobados/reprobados
while + mayor	Ej. 6 — mayor de 5 nums	Desafío 1: mejor y peor nota
switch	Ej. 7 — menú de opciones	Desafío 2: conversor de unidades
do...while + validar	Ej. 8 — número > 0	Desafíos 1 y 3: validar entradas
do...while + switch	Ej. 9 — menú repetido	Desafío 3: menú de operaciones
isNaN + return null	Coloquio — obtenerNumero()	Los tres desafíos: función de validación

Desafío 1 — Calculadora de Rendimiento Académico

Intermedio · 35 min

Crear una página web que permita al alumno registrar sus materias y notas una por una, y que calcule automáticamente su rendimiento general cada vez que agrega una materia.

¿Por qué esta aplicación? Es la versión HTML real de los ejercicios de teoría: en lugar de hardcodear los números en el código, el usuario los ingresa desde la página. En lugar de imprimir en consola, los resultados se muestran en el DOM.

Estructura HTML requerida:

- Un *input* de texto para el nombre de la materia (id: "materia").
- Un *input* numérico para la nota del 0 al 10 (id: "nota").
- Un botón "Agregar materia" con *onclick* que llame a *agregarMateria()*.
- Un *div* con id "lista" donde se mostrará la tabla de materias registradas.
- Un *div* con id "resumen" donde se mostrarán las estadísticas.

Funciones JavaScript a implementar:

1. obtenerDatos() — Validación de entrada

- Leer el valor del *input* "materia": no puede estar vacío.
- Leer el valor del *input* "nota": usar *isNaN()* para validar que sea un número.
- Validar que la nota esté entre 0 y 10. Si no lo está, mostrar error y retornar *null*.
- Si todo es válido, retornar un objeto con { *materia*, *nota* }.

Concepto aplicado: *do...while para validar entrada · isNaN · return null (patrón del coloquio)*

2. clasificarNota(nota) — Recibe un número y devuelve la categoría

- Si *nota* ≥ 9 : devolver "Sobresaliente".
- Si *nota* ≥ 7 : devolver "Bueno".
- Si *nota* ≥ 6 : devolver "Regular".
- Si *nota* ≥ 4 : devolver "Aprobado mínimo".
- Si *nota* < 4 : devolver "Insuficiente".

Concepto aplicado: *if / else if / else encadenados (Ejercicio 1 de teoría)*

3. agregarMateria() — Función principal conectada al botón

- Llamar a *obtenerDatos()*. Si retorna *null*, no continuar.
- Guardar la materia y nota en un **array global** llamado *materias*.
- Llamar a *mostrarLista()* y luego a *calcularResumen()*.
- Limpiar los *inputs* después de agregar.

4. mostrarLista() — Genera la tabla en el DOM

- Usar un bucle *for* para recorrer el array *materias*.
- Por cada materia, construir una fila HTML con el nombre, la nota y la categoría (llamando a *clasificarNota()*).
- Mostrar el HTML resultante en el *div* "lista" usando *innerHTML*.

Concepto aplicado: *Bucle for para recorrer una lista · innerHTML dinámico (Ejercicio 2 de teoría)*

5. calcularResumen() — Calcula y muestra las estadísticas

- Usar un bucle *while* con una variable *suma = 0* para acumular todas las notas (Ejercicio 3).
- Usar un contador para contar cuántas materias tienen nota ≥ 6 (aprobadas) y cuántas no (Ejercicio 4).
- Encontrar la nota más alta y la más baja usando comparaciones sucesivas (Ejercicio 6).
- Calcular el promedio dividiendo la suma por la cantidad de materias.
- Mostrar todo en el div *"resumen"*: promedio, aprobadas, reprobadas, mejor nota, peor nota.

Concepto aplicado: *while + acumulador · contador · búsqueda del mayor/menor (Ejercicios 3, 4 y 6)*

Ejemplo de salida esperada en el resumen (con 4 materias ingresadas):

Promedio general: 7.25 | Aprobadas: 3 | Reprobadas: 1 | Mejor nota: 9 (Matemáticas) |
Peor nota: 3 (Historia)

Punto extra: Agregar un botón "Limpiar todo" que vacíe el array *materias*, limpie el DOM y reinicie el resumen.

Desafío 2 — Conversor de Unidades con Menú

Intermedio · 25 min

Crear una página web que funcione como conversor de unidades. El usuario elige qué tipo de conversión quiere hacer desde un menú desplegable y luego ingresa el valor a convertir.

¿Por qué esta aplicación? El switch del ejercicio de teoría era un menú estático hardcoded. Acá el menú es un `<select>` real que el usuario controla, y el switch decide qué fórmula aplicar según la opción elegida.

Conversiones disponibles:

Opción en el select	Conversión	Fórmula
1	Kilómetros → Millas	$\text{millas} = \text{km} \times 0.621371$
2	Millas → Kilómetros	$\text{km} = \text{millas} \times 1.60934$
3	Kilogramos → Libras	$\text{libras} = \text{kg} \times 2.20462$
4	Libras → Kilogramos	$\text{kg} = \text{libras} \times 0.453592$
5	Celsius → Fahrenheit	$F = (C \times 9/5) + 32$
6	Fahrenheit → Celsius	$C = (F - 32) \times 5/9$
7	Metros → Pies	$\text{pies} = \text{metros} \times 3.28084$

Estructura HTML requerida:

- Un `<select>` con las 7 opciones de conversión (id: "tipoConversion").
- Un `input` numérico para el valor a convertir (id: "valor").
- Un botón "Convertir" con `onclick="convertir()"`.
- Un `div` (id: "resultado") donde se muestra la respuesta.

Funciones JavaScript a implementar:**1. obtenerNumero()** — igual al patrón del coloquio:

- Leer el input "valor", validar con `isNaN()` que sea número y que no esté vacío.
- Retornar `null` si hay error, o el número convertido a `parseFloat` si es válido.

2. convertir() — función principal:

- Llamar a `obtenerNumero()`. Si retorna `null`, no continuar.
- Leer el valor del `<select>` con `getElementById("tipoConversion").value`.
- Usar un `switch` con los 7 casos para aplicar la fórmula correspondiente.
- En cada `case`: calcular el resultado y guardarlo en una variable `resultado`.
- Al finalizar el `switch`, mostrar el resultado en el `div` con un mensaje claro.
- Usar `toFixed(2)` para mostrar el resultado con 2 decimales.

Concepto aplicado: `switch` con 7 casos · `break` · `default` (Ejercicios 7 y 9 de teoría)

Ejemplo de salida esperada: Si el usuario elige opción 5 ($^{\circ}\text{C} \rightarrow ^{\circ}\text{F}$) e ingresa 100, el `div resultado` debe mostrar: "100 $^{\circ}\text{C}$ equivalen a 212.00 $^{\circ}\text{F}$ ".

Punto extra: Agregar un **historial de conversiones**. Cada vez que el usuario convierte un valor, agregar el resultado al final de una lista visible en la página. La lista se construye con un bucle for sobre un array global `historial[]`.

Desafío 3 — Registrador de Números con Estadísticas

Crear una página web que permita al usuario ingresar números uno por uno. La aplicación los va registrando y muestra estadísticas actualizadas en tiempo real. El usuario puede seguir ingresando o detener el registro cuando quiera.

¿Por qué esta aplicación? Los ejercicios 3, 4, 5 y 6 de teoría hacían cálculos sobre series de números con bucles. Aquí cada vez que el usuario hace clic en "Agregar", es como si el bucle avanzara un paso. Las estadísticas se recalculan con un `while` sobre el array cada vez.

Estructura HTML requerida:

- Un `input` numérico para ingresar cada número (id: "numero").
- Un botón "Agregar número" — llama a `agregarNumero()`.
- Un botón "Ver tabla de multiplicar" — llama a `tablaMultiplicar()`.
- Un botón "Reiniciar" — llama a `reiniciar()`.
- Un `div` (id: "listaNumeros") — muestra los números ingresados.
- Un `div` (id: "estadisticas") — muestra las estadísticas.
- Un `div` (id: "tabla") — muestra la tabla de multiplicar.

Funciones JavaScript a implementar:

1. `agregarNumero()`

- Validar con `isNaN()` que el input contenga un número. Si no, mostrar error.
- Agregar el número al array global `numeros[]`.
- Llamar a `mostrarNumeros()` y a `calcularEstadisticas()`.
- Limpiar el input y poner el foco de nuevo en él para que el usuario pueda seguir rápido.

2. `mostrarNumeros()`

- Usar un bucle `for` para mostrar todos los números del array en el `div` "listaNumeros", separados por comas.
- Indicar cuántos números hay en total.

Concepto aplicado: Bucle `for` sobre un array (Ejercicio 2 de teoría)

3. `calcularEstadisticas()`

- Usando un bucle `while`, calcular la **suma** de todos los números del array.
- Calcular el **promedio** dividiendo la suma por la cantidad de elementos.
- Usando el mismo recorrido, contar cuántos son **positivos** y cuántos son **negativos**.
- Usando comparaciones, encontrar el **mayor** y el **menor** del array.
- Mostrar todo en el `div` "estadisticas": suma, promedio, positivos, negativos, mayor y menor.

Concepto aplicado: `while` + acumulador · contador · mayor/menor (Ejercicios 3, 4 y 6)

4. `tablaMultiplicar()`

- Pedir al usuario que elija un número del array (puede ser el último ingresado).
- Usar un bucle `for` del 1 al 10 para generar la tabla de multiplicar de ese número.
- Mostrar cada línea en el `div` "tabla". Ej: "7 × 3 = 21".

Concepto aplicado: Bucle `for` + concatenación de strings (Ejercicio 5 de teoría)

5. reiniciar()

- Vaciar el array `numeros[]`.
- Limpiar el contenido de los tres divs.
- Limpiar el input.

Punto extra: Antes de agregar cada número, validar con un `do...while` que el usuario ingrese un número válido. Mostrar un mensaje de error en rojo debajo del input si el valor no es número, y no agregarlo al array hasta que sea correcto.

Criterios de evaluación

Criterio	Desafíos	Pts
Estructura HTML correcta (IDs, botones, divs, sin <code>alert()</code>)	Todos	15
Función <code>obtenerDatos/obtenerNumero</code> con <code>isNaN</code> y <code>return null</code>	Todos	15
<code>if / else if / else</code> para clasificar o decidir	1 y 2	15
<code>switch</code> con casos y <code>break</code>	2	15
Bucle <code>for</code> para mostrar listas en el DOM	1 y 3	15
<code>while</code> con acumulador, contador y mayor/menor	1 y 3	15
Funciones separadas por responsabilidad	Todos	10
Puntos extra (historial, limpiar, <code>do...while</code> de validación)	Todos	+10

Esta práctica no tiene entrega obligatoria. Se recomienda resolver los desafíos en orden y releer los ejercicios de teoría cuando no recuerdes cómo implementar algún concepto. La clave está en reconocer que la lógica es la misma — solo cambia que ahora los datos vienen de un input y los resultados van al DOM.